

ProbOS — Month 1, Week 3

Monte Carlo Engine, Sobol Sensitivity, Provenance

Nisong Monyimba

July 3, 2026

Week 3 goal

Build the core inference-adjacent kernel: **MonteCarloEngine** for vectorised uncertainty propagation, **SobolSensitivity** for variance-based sensitivity analysis, and **ProvenanceTracker** for causal audit trails – all validated against **BatteryModel2Cell** from Week 2.

Day-by-day plan

Day 1 – MonteCarloEngine core

- Draw N parameter sets from priors, propagate all particles through `forward_batch()` over `n.steps`
- Track P05/P50/P95 percentile trajectories at every timestep
- `python/src/monte_carlo.py` skeleton and core loop

Day 2 – Convergence and CLT verification

- `sigma/sqrt(N)` convergence certificate
- Verify empirically that MC error shrinks at the theoretical $1/\sqrt{N}$ rate on **BatteryModel2Cell**
- `python/examples/week3_mc_battery.py`

Day 3 – SobolSensitivity

- First-order (S_1) and total-effect (S_T) Sobol indices via SALib's Saltelli sampling scheme
- `python/src/sensitivity.py`
- Document the known SALib reproducibility limitation (results vary run-to-run at low sample sizes) rather than hiding it

Day 4 – Sobol validation on BatteryModel2Cell

- Run full sensitivity analysis: identify **Ea_SEI** as the dominant parameter for thermal runaway ($S_1 = 0.457$)
- `python/examples/week3_sobol_battery.py`

Day 5 – ProvenanceTracker

- Causal DAG: which particles and which parameter draws produced which outcomes
- Query interface: trace any output value back to its causing inputs
- `python/src/provenance.py`

Day 6 – Integration across all three components

- Confirm `MonteCarloEngine`, `SobolSensitivity`, and `ProvenanceTracker` all operate correctly together on the same `BatteryModel2Cell` + priors combination
- Cross-check: does the provenance trail correctly identify `Ea_SEI` as a major driver, consistent with the Sobol result?

Day 7 – Test coverage + hardening

- `python/tests/test_monte_carlo.py`, `test_sensitivity.py`, `test_provenance.py`
- mypy strict, ruff clean, CI green

What was actually accomplished (retrospective)

- `MonteCarloEngine` implemented exactly as planned, with P05/P50/P95 tracking and $\sigma/\sqrt{N}$ convergence certificate
- CLT convergence verified empirically on `BatteryModel2Cell`
- `SobolSensitivity` implemented via SALib; `Ea_SEI` confirmed as the dominant thermal-runaway driver ($S_1 = 0.457$, $S_T = 0.729$)
- SALib's known reproducibility limitation documented directly in code comments and tests rather than glossed over
- `ProvenanceTracker` causal DAG implemented and validated against the Sobol result – consistent story across both tools
- 91 new Python tests across the three new modules

Numbers at Week 3 close

- Python tests: 202/202 passing (111 Weeks 1–2 + 91 new)
- C++ tests: 13/13 passing (unchanged)
- mypy strict: 0 errors
- ruff: 0 warnings

Carried into Week 4

With the Python kernel complete (`Distribution`, `Model`, `MonteCarloEngine`, `SobolSensitivity`, `ProvenanceTracker`), Week 4 turns to performance (DP profiling, C++/OpenMP), language (PDSL compiler v0.1), and breadth (cross-discipline examples proving the kernel generalises beyond batteries).